Lifecycle Order Flowchart

The hooks a page author writes, in the order they fire. Only those: no change-detection passes, no router events. Measured, not quoted from docs.

Angular · Ionic page hook · Guard or resolver · Old hook, re-created by the shim · First visible render

awaits Promise the framework awaits what you return (Ionic 3: a Promise; the Angular router: a Promise or an Observable) and navigation pauses until it settles. Load or check data here.

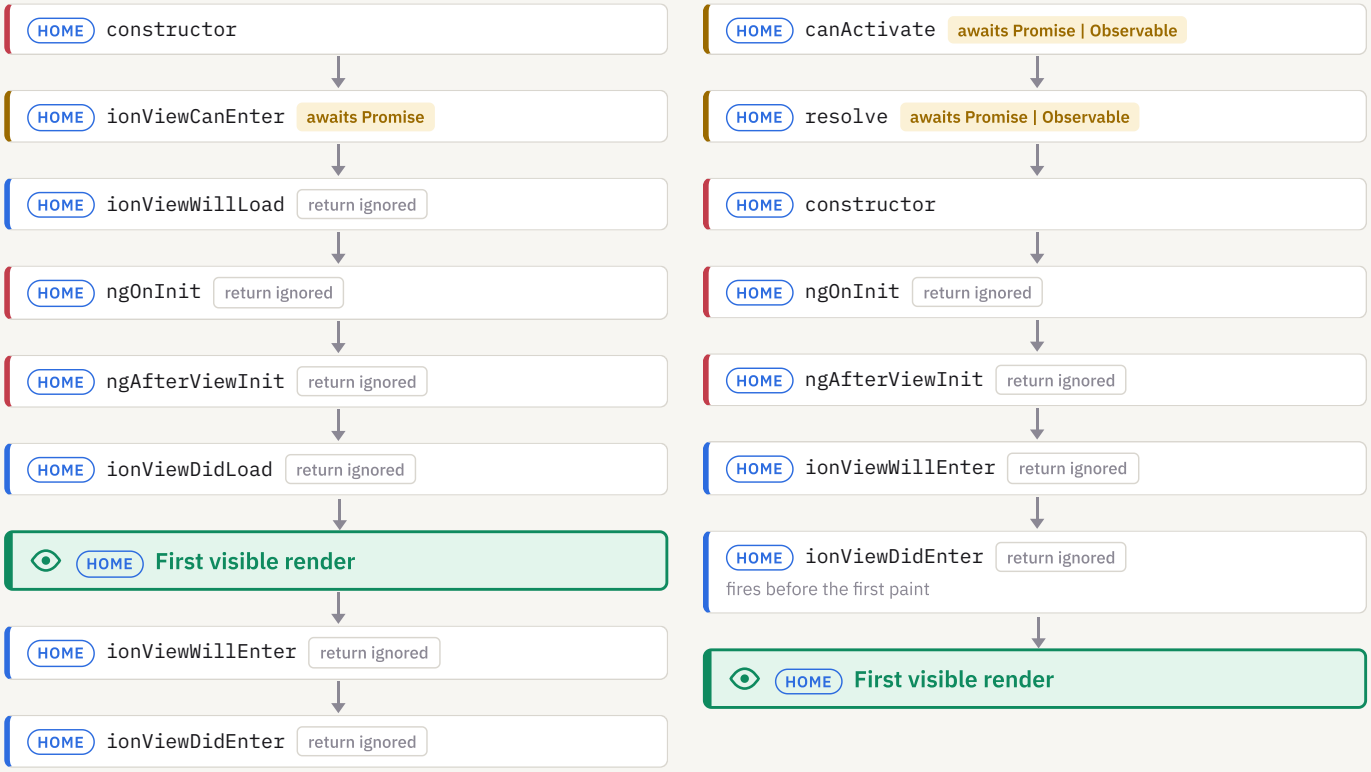
return ignored you can write the hook as `async`, but nothing waits for it; show your own loading state

---

1 Old vs new

Left Ionic 3.9.5 / Angular 5.0.3. Right Ionic 8.4.3 / Angular 20.1.3.

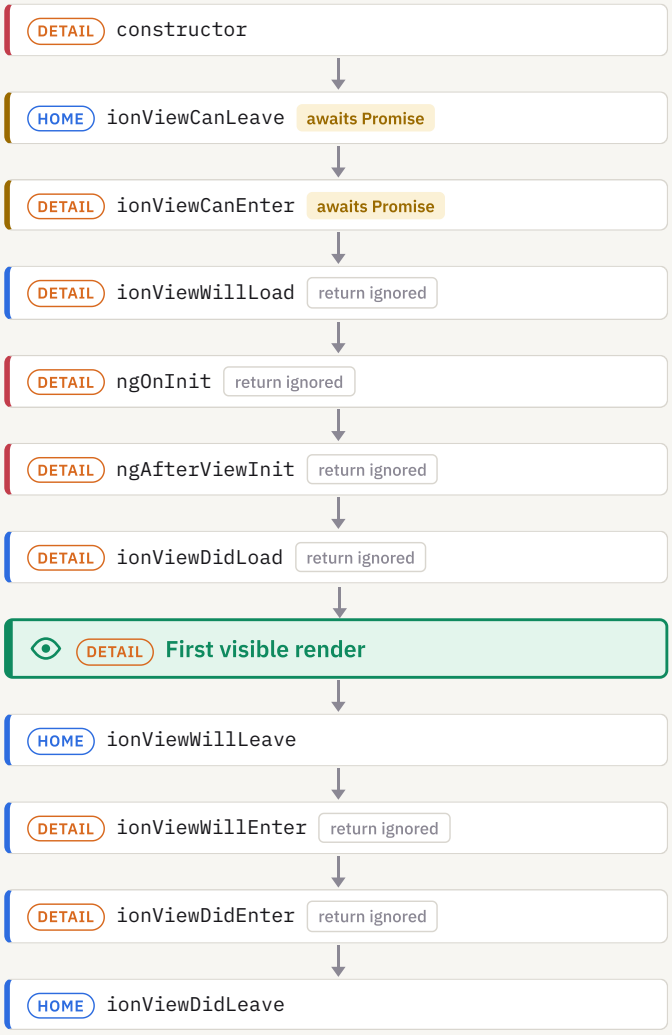
App start



Push Home → Detail

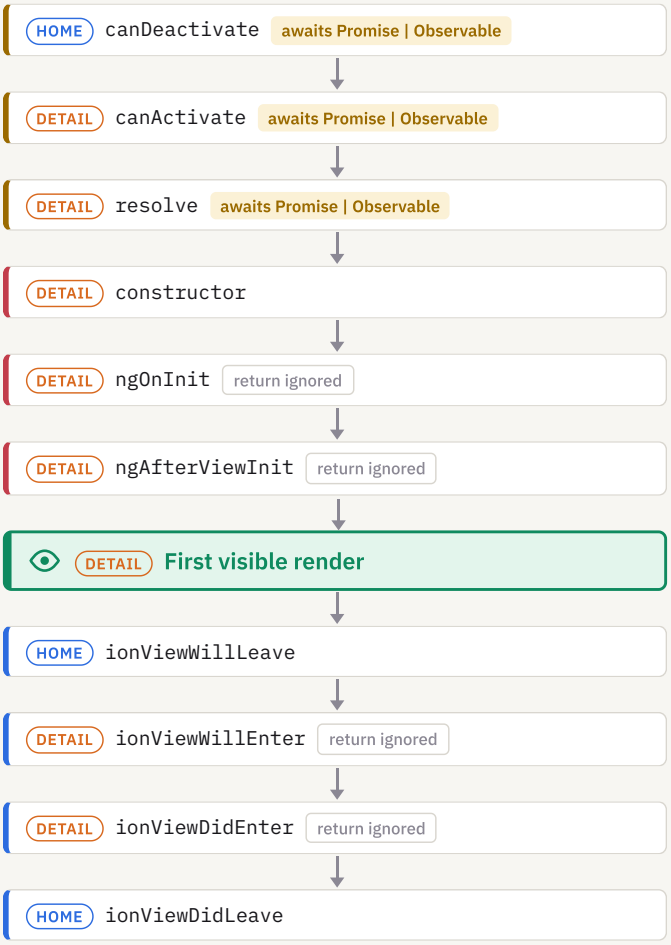
Old

Ionic 3.9.5 / Angular 5.0.3



New

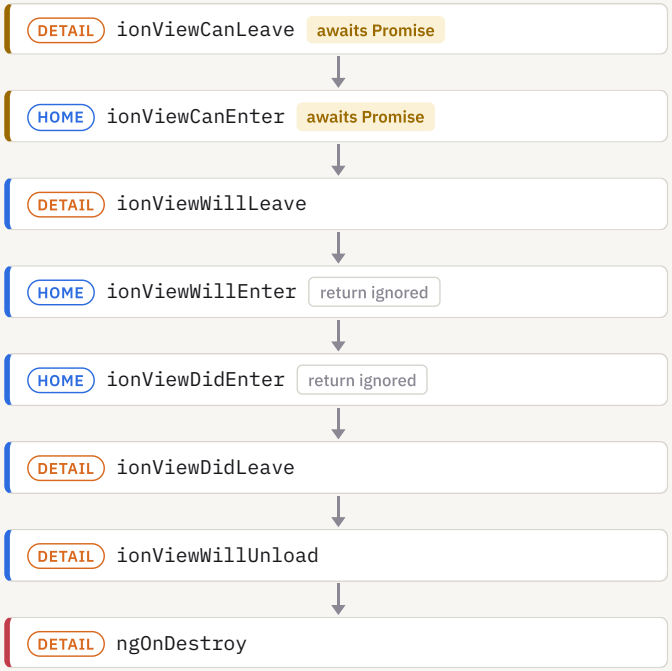
Ionic 8.4.3 / Angular 20.1.3



Pop back to Home

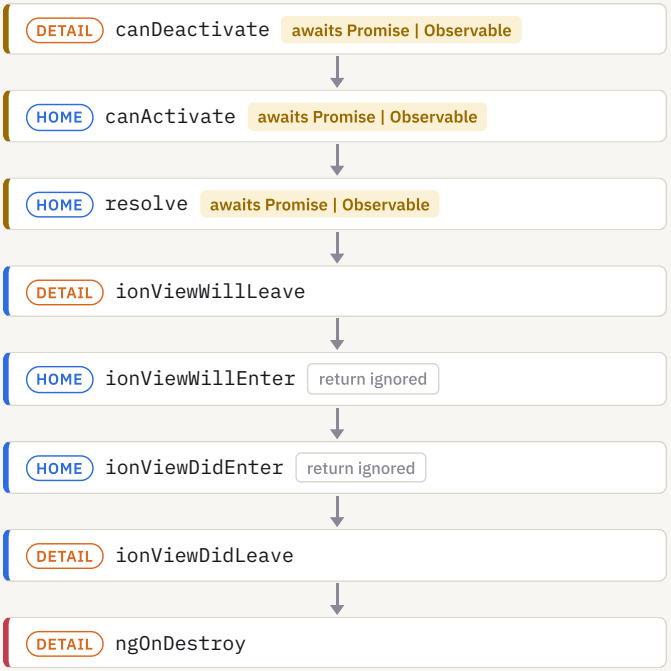
Old

Ionic 3.9.5 / Angular 5.0.3



New

Ionic 8.4.3 / Angular 20.1.3



Where you can wait for data

Only the amber-tagged hooks are awaited: Ionic 3 awaited a returned Promise, the Angular router awaits a Promise or an Observable. Everything else runs and the app moves on, so data loaded there needs its own loading state.

YOU WANT TO	IONIC 3	IONIC 8 + ANGULAR 20	PAGE EXISTS WHEN IT RUNS?
Block entering until data or permission is ready	`ionViewCanEnter`	`canActivate` for a yes/no, `resolve` for data; both await a Promise or Observable (`canMatch` too)	Old: yes, constructed first. New: no.
Block leaving (unsaved changes)	`ionViewCanLeave`	`canDeactivate`	Yes in both.
Show the page, then load	`ionViewDidLoad` or `ionViewWillEnter`	`ngOnInit` or `ionViewWillEnter`, not awaited	Yes in both.
Refresh every time the page is shown again	`ionViewWillEnter`	`ionViewWillEnter`, not awaited	Yes in both; the page is reused on pop.

2 How the shim is called

Two route guards and one base class. The guards turn router calls into the old guard methods; the base class turns Angular hooks into the old load and unload hooks. The four enter and leave hooks need nothing: Ionic 8 still calls them by the same names.



ROUTE

```
{
  path: 'detail',
  component: DetailPage,
  canActivate:   [legacyCanEnter(DetailPage)],
  canDeactivate: [legacyCanLeave],
}
```

GUARDS

```
export const legacyCanLeave: CanDeactivateFn<LegacyPage> =
  (page) => page?.ionViewCanLeave?.() ?? true;

export function legacyCanEnter(Page): CanActivateFn {
  return (route, state) =>
    Page.ionViewCanEnter ? Page.ionViewCanEnter(route, state)
}
```

BASE CLASS

```
@Directive()
export abstract class LegacyPage {
  static ionViewCanEnter?(route, state): boolean | Promise<bool

  ngOnInit()        { this.ionViewWillLoad(); }
  ngAfterViewInit() { this.ionViewDidLoad(); }
  ngOnDestroy()     { this.ionViewWillUnload(); }

  ionViewCanLeave(): boolean | Promise<boolean> { return true;
  ionViewWillLoad() {}  ionViewDidLoad() {}  ionViewWillUnload
}
```

A PAGE

```
export class DetailPage extends LegacyPage {
  static override ionViewCanEnter(route, state) {
    return inject(AuthService).isAllowed();   // no this here
  }
  override ionViewCanLeave() { return !this.form.dirty; }
  override ionViewDidLoad()  { /* DOM exists, same as Ionic 3
}
```

The one thing that does not come back is an `ionViewCanEnter` that reads instance state. The router decides before the page exists, so the check has to be static or live in a service. If it must see the instance, let the guard pass and check in `ionViewWillEnter`, then navigate back; that restores "construct first, then decide" at the cost of a possible flash.

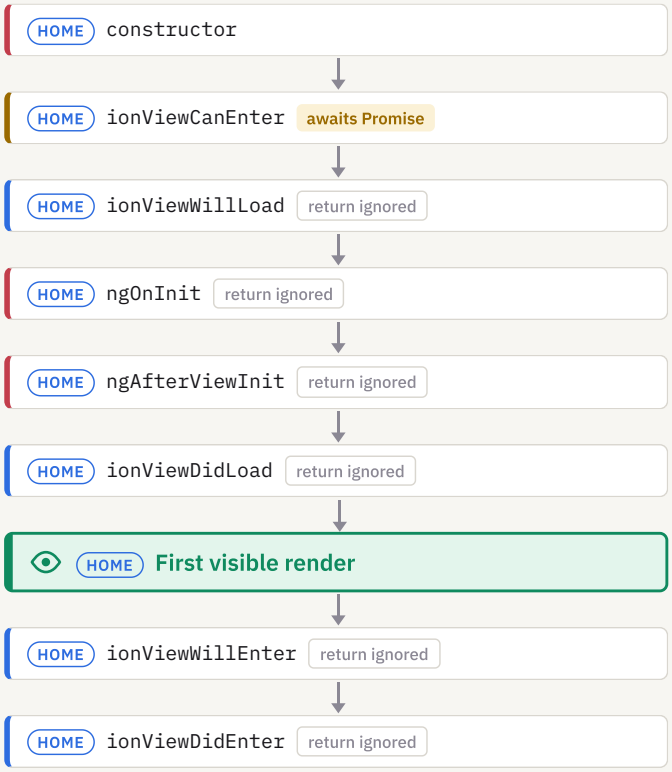
3 Old vs new with the shim

Same phases, with the shim wired in. A box without a tag is in the same place as Ionic 3.
Dashed red boxes cannot be moved back; the reason is under each.

App start

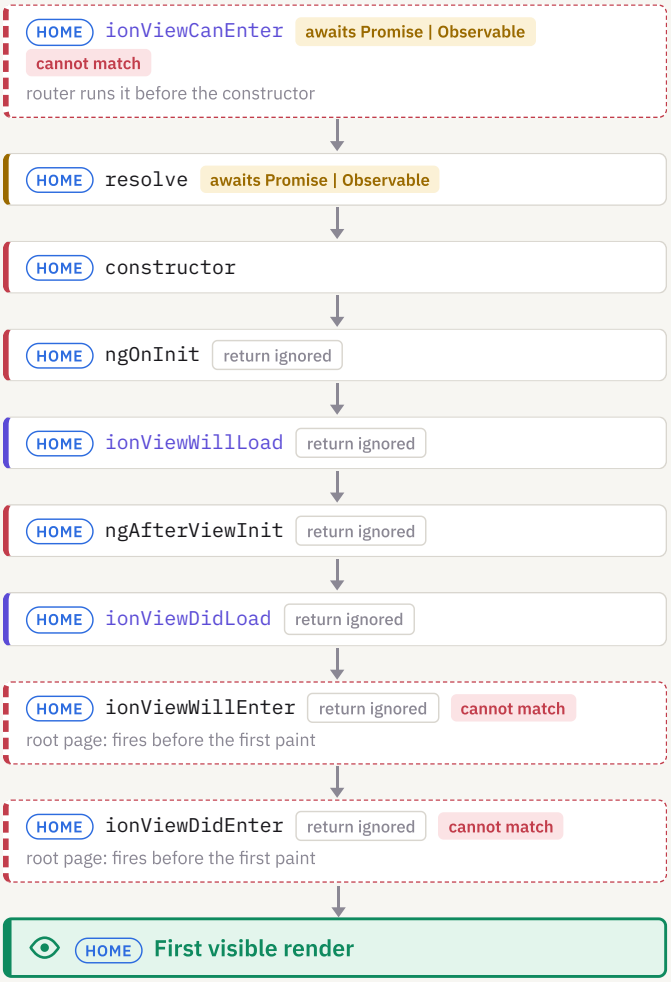
Old

Ionic 3.9.5 / Angular 5.0.3



New + shim

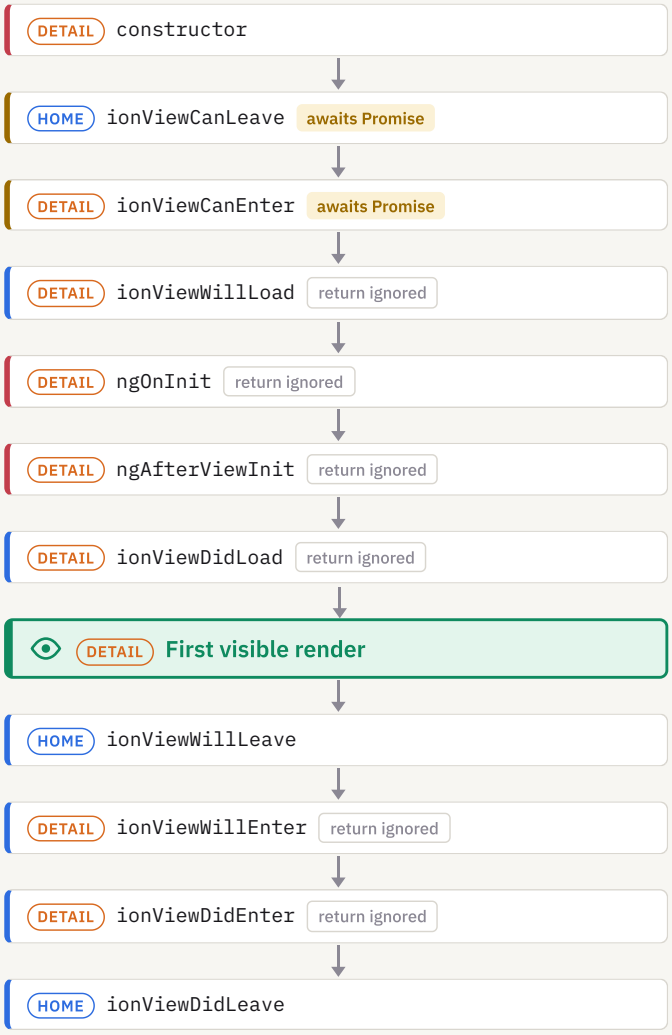
Ionic 8.4.3 / Angular 20.1.3 + compat shim



Push Home → Detail

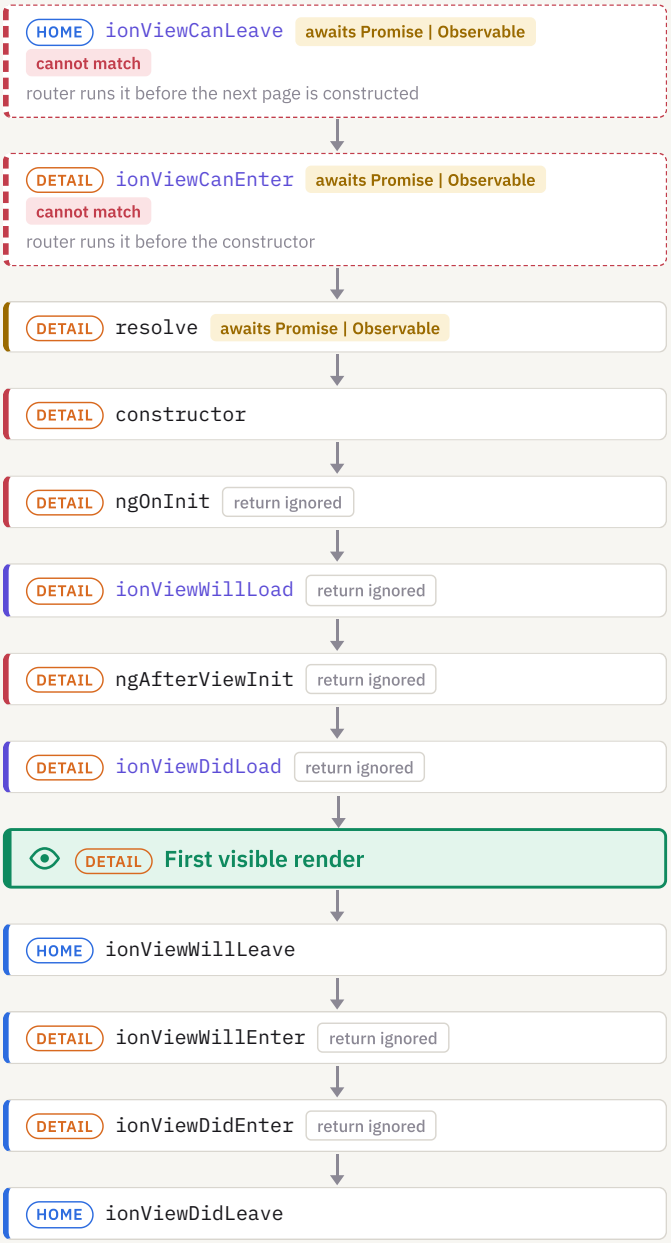
Old

Ionic 3.9.5 / Angular 5.0.3



New + shim

Ionic 8.4.3 / Angular 20.1.3 + compat shim



Pop back to Home

Old

Ionic 3.9.5 / Angular 5.0.3

DETAIL ionViewCanLeave — awaits Promise

↓

HOME ionViewCanEnter — awaits Promise

↓

DETAIL ionViewWillLeave

↓

HOME ionViewWillEnter — return ignored

↓

HOME ionViewDidEnter — return ignored

↓

DETAIL ionViewDidLeave

↓

DETAIL ionViewWillUnload

↓

DETAIL ngOnDestroy

New + shim

Ionic 8.4.3 / Angular 20.1.3 + compat shim

DETAIL ionViewCanLeave — awaits Promise | Observable

↓

HOME ionViewCanEnter — awaits Promise | Observable

static, no instance yet

↓

HOME resolve — awaits Promise | Observable

↓

DETAIL ionViewWillLeave

↓

HOME ionViewWillEnter — return ignored

↓

HOME ionViewDidEnter — return ignored

↓

DETAIL ionViewDidLeave

↓

DETAIL ngOnDestroy

↓

DETAIL ionViewWillUnload

First visible render

There is no "painted" event in either stack. The green box is the earliest point where the page's DOM is complete and the next frame will show it: a `requestAnimationFrame` after `ngAfterViewInit` in the old app, `afterNextRender` in the new one. On a push the page then slides in and is fully visible at `ionViewDidEnter`. On app start the root page has nothing to leave, so Ionic 8 fires both enter hooks at once, before anything is painted.